

Язык программирования C++

Лекция 13: наследование, полиморфизм, стандартная библиотека, шаблоны

Никита Лисица

2026

Множественное наследование

```
class Readable {  
public:  
    virtual void read(char* dst, size_t bytes) = 0;  
};  
  
class Writable {  
public:  
    virtual void write(const char* src, size_t bytes) = 0;  
};
```

Множественное наследование

```
class Readable {  
public:  
    virtual void read(char* dst, size_t bytes) = 0;  
};  
  
class Writable {  
public:  
    virtual void write(const char* src, size_t bytes) = 0;  
};
```

- Хотим сделать класс, который является и Readable, и Writable, но как?

Множественное наследование

- С помощью множественного наследования (*multiple inheritance*):

```
class File : public Readable, public Writable {  
public:  
    void read(char* dst, size_t bytes) override { ... }  
    void write(const char* src, size_t bytes) override { ... }  
};
```

Множественное наследование

- С помощью *множественного наследования (multiple inheritance)*:

```
class File : public Readable, public Writable {  
public:  
    void read(char* dst, size_t bytes) override { ... }  
    void write(const char* src, size_t bytes) override { ... }  
};
```

- Такой класс содержит все поля и методы всех классов, от которых наследуется

Множественное наследование

- С помощью *множественного наследования (multiple inheritance)*:

```
class File : public Readable, public Writable {  
public:  
    void read(char* dst, size_t bytes) override { ... }  
    void write(const char* src, size_t bytes) override { ... }  
};
```

- Такой класс содержит все поля и методы всех классов, от которых наследуется
- Указатель (или ссылка) такой класс приводится к типу указателя (или ссылки) на любой из базовых классов

Множественное наследование

- Что, если у базовых классов есть одинаковые поля/методы?

```
class Readable {  
protected:  
    size_t totalBytes;  
public:  
    ...  
};
```

```
class Writable {  
protected:  
    size_t totalBytes;  
public:  
    ...  
};
```

Множественное наследование

- Что, если у базовых классов есть одинаковые поля/методы?

```
class Readable {  
protected:  
    size_t totalBytes;  
public:  
    ...  
};
```

```
class Writable {  
protected:  
    size_t totalBytes;  
public:  
    ...  
};
```

- Можно использовать явное разрешение имён:

```
class File : public Readable, public Writable {  
public:  
    size_t bytesRead() const { return Readable::totalBytes; }  
    size_t bytesWritten() const { return Writable::totalBytes; }  
};
```

Виртуальное наследование

- Что будет, если оба базовых класса сами наследуются от одного и того же класса (т.н. *diamond problem*)?

```
class IOBase {  
protected:  
    size_t totalBytes;  
};  
  
class Readable : public IOBase {  
    ...  
};  
  
class Writable : public IOBase {  
    ...  
};  
  
class File : public Readable, public Writable {  
    ...  
};
```

- У обоих базовых классов будет своя копия подобиъекта **IOBase**

- У обоих базовых классов будет своя копия подобъекта `IOBase`
- $\implies$ В классе `File` содержатся два подобъекта типа `IOBase`

- У обоих базовых классов будет своя копия подобъекта **IOBase**
- $\implies$ В классе **File** содержатся два подобъекта типа **IOBase**
- Иногда это ровно то, что нужно; а иногда хочется, чтобы подобъект общего типа был один

Виртуальное наследование

- У обоих базовых классов будет своя копия подобъекта `IOBase`
- $\implies$ В классе `File` содержатся два подобъекта типа `IOBase`
- Иногда это ровно то, что нужно; а иногда хочется, чтобы подобъект общего типа был один
- Решение: *виртуальное наследование (virtual inheritance)*:

```
class Readable : public virtual IOBase {  
    ...  
};  
  
class Writable : public virtual IOBase {  
    ...  
};
```

- При виртуальном наследовании подобъект типа `IOBase` будет только один

- При виртуальном наследовании подобъект типа `IOBase` будет только один
- Почему это свойство классов `Readable` и `Writable`, а не класса `File`?

- При виртуальном наследовании подобъект типа `IOBase` будет только один
- Почему это свойство классов `Readable` и `Writable`, а не класса `File`?
- `Readable`/`Writable` не могут независимо хранить подобъект `IOBase`, и вместо этого хранят *указатель* на подобъект `IOBase`

- При виртуальном наследовании подобъект типа `IOBase` будет только один
- Почему это свойство классов `Readable` и `Writable`, а не класса `File`?
- `Readable`/`Writable` не могут независимо хранить подобъект `IOBase`, и вместо этого хранят *указатель* на подобъект `IOBase`
- Сам подобъект `IOBase` кладётся где-то по усмотрению компилятора, часто – в конце класса

Виртуальное наследование

- Расположение данных в памяти получится аналогичным вот такому коду:

```
class IOBase { ... };

class Readable {
    // Аналог public virtual IOBase
    IOBase* ioBasePtr;
};

class Writable {
    // Аналог public virtual IOBase
    IOBase* ioBasePtr;
};

class File : public Readable, public Writable {
    // Компилятор сам положит подобъект IOBase
    // и правильно запишет указатели ioBasePtr
    IOBase ioBase;
}
```

Виртуальное наследование

- Конструктор виртуального базового класса должен явно вызывать любой класс, наследующийся от него (напрямую или через другие классы):

```
class IOBase {
public:
    IOBase(int flags);
};

class Readable : public virtual IOBase {
public:
    Readable() : IOBase(READ_FLAG) {}
};

class Writable : public virtual IOBase {
    Writable() : IOBase(WRITE_FLAG) {}
};

class File : public Readable, public Writable {
public:
    File() : IOBase(READ_FLAG | WRITE_FLAG) {}
};
```

- ООП-языки часто отдельно выделяют *интерфейсы*: спецификации наборов методов, которые должен поддерживать некоторый класс

Интерфейсы

- ООП-языки часто отдельно выделяют *интерфейсы*: спецификации наборов методов, которые должен поддерживать некоторый класс
- В C++ роль интерфейсов выполняют классы с *чисто виртуальными методами и без полей*:

```
class Hashable {
public:
    virtual size_t getHash() = 0;

    // Не забываем про виртуальный деструктор!
    virtual ~Hashable() {}
};

class Comparable {
public:
    virtual bool equals(Comparable* other) = 0;

    // Не забываем про виртуальный деструктор!
    virtual ~Comparable() {}
};
```

- Обычно лучше использовать множественное наследование только от интерфейсов – некоторые ООП-языки (Java, C#) разрешают только такое множественное наследование

- Обычно лучше использовать множественное наследование только от интерфейсов – некоторые ООП-языки (Java, C#) разрешают только такое множественное наследование
- Виртуальное наследование – подход C++ к решению diamond problem, в других языках есть другие решения, см.
en.wikipedia.org/wiki/Multiple_inheritance#Mitigation

- В общем случае приведение указателя на класс-наследник к указателю на базовый класс – нетривиальная операция

Приведение указателей

- В общем случае приведение указателя на класс-наследник к указателю на базовый класс – нетривиальная операция
- При обычном наследовании часто базовый класс лежит в начале объекта класса-наследника, и приведение указателей не меняет адрес

Приведение указателей

- В общем случае приведение указателя на класс-наследник к указателю на базовый класс – нетривиальная операция
- При обычном наследовании часто базовый класс лежит в начале объекта класса-наследника, и приведение указателей не меняет адрес
- При множественном наследовании может потребоваться сдвинуть указатель

Приведение указателей

- В общем случае приведение указателя на класс-наследник к указателю на базовый класс – нетривиальная операция
- При обычном наследовании часто базовый класс лежит в начале объекта класса-наследника, и приведение указателей не меняет адрес
- При множественном наследовании может потребоваться сдвинуть указатель
- При виртуальном наследовании может потребоваться прочесть из памяти адрес, по которому лежит объект базового класса

- `Derived*` всегда (при публичном наследовании) приводится к `Base*`

Приведение указателей

- `Derived*` всегда (при публичном наследовании) приводится к `Base*`
- `Derived**` автоматически **не приводится** к `Base**`: для этого нужно пройтись по массиву указателей и привести каждый отдельно (это может быть нетривиальная операция)

Конфликты имён

- Часто возникает ситуация, когда в разных частях программы появились функции/классы с одинаковыми именами:

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
class List {  
    Node* head;  
    ...  
};
```

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

```
class Tree {  
    Node* root;  
    ...  
};
```

Конфликты имён

- Часто возникает ситуация, когда в разных частях программы появились функции/классы с одинаковыми именами:

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
class List {  
    Node* head;  
    ...  
};
```

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

```
class Tree {  
    Node* root;  
    ...  
};
```

- Это приведёт к ошибкам компиляции, если какой-то код использует и `List`, и `Tree`

Конфликты имён

- Часто возникает ситуация, когда в разных частях программы появились функции/классы с одинаковыми именами:

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
class List {  
    Node* head;  
    ...  
};
```

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
};
```

```
class Tree {  
    Node* root;  
    ...  
};
```

- Это приведёт к ошибкам компиляции, если какой-то код использует и `List`, и `Tree`
- (Если невезёт, может обойтись без ошибок компиляции, но с runtime ошибками!)

Конфликты имён

- Как избежать конфликтов имён? Вариант 1: префиксы в именах

```
struct ListNode {  
    int data;  
    ListNode* next;  
};
```

```
struct TreeNode {  
    int data;  
    TreeNode* left;  
    TreeNode* right;  
};
```

Конфликты имён

- Как избежать конфликтов имён? Вариант 1: префиксы в именах

```
struct ListNode {  
    int data;  
    ListNode* next;  
};
```

```
struct TreeNode {  
    int data;  
    TreeNode* left;  
    TreeNode* right;  
};
```

- Длиннее имена функций/структур/классов, сложнее читать
- Переводит проблему на уровень выше: кто-то мог создать два разных класса **Tree** :)

Конфликты имён

- Как избежать конфликтов имён? Вариант 2: вложенные типы

```
class List {  
    struct Node {  
        int data;  
        Node* next;  
    };  
  
    Node* head;  
};
```

```
class Tree {  
    struct Node {  
        int data;  
        Node* left;  
        Node* right;  
    };  
  
    Node* root;  
};
```

Конфликты имён

- Как избежать конфликтов имён? Вариант 2: вложенные типы

```
class List {  
    struct Node {  
        int data;  
        Node* next;  
    };  
  
    Node* head;  
};
```

```
class Tree {  
    struct Node {  
        int data;  
        Node* left;  
        Node* right;  
    };  
  
    Node* root;  
};
```

- Отлично подходит в этом конкретном примере
- Инкапсуляция: делает и сам тип **Node** недоступным снаружи
- Не подходит, если нет класса, который логически должен содержать наши функции/классы

Пространства имён

- Как избежать конфликтов имён? Вариант 3: *пространства имён (namespaces)*

```
namespace xml {  
    class Node { ... };  
    class Parser { ... };  
}  
  
namespace json {  
    class Node { ... };  
    class Parser { ... };  
}  
  
json::Parser parser("myFile.json");  
...
```

Пространства имён

- Как избежать конфликтов имён? Вариант 3: *пространства имён (namespaces)*

```
namespace xml {  
    class Node { ... };  
    class Parser { ... };  
}  
  
namespace json {  
    class Node { ... };  
    class Parser { ... };  
}  
  
json::Parser parser("myFile.json");  
...
```

- При компиляции пространства имён тоже используются для name mangling'a

Пространства имён

- Пространство имён можно разбить на несколько файлов:

```
// xml/node.h

namespace xml {

    class Node { ... };

}
```

```
// xml/parser.h

namespace xml {

    class Parser { ... };

}
```

```
// xml/node.cpp

namespace xml {

    Node::Node() { ... }

}
```

```
// xml/parser.cpp

namespace xml {

    Parser::Parser() { ... }

}
```

- Если пространство имён используется часто, можно подключить его с помощью конструкции `using namespace`:

```
using namespace xml;  
Parser parser("myFile.xml");
```

Пространства имён

- Если пространство имён используется часто, можно подключить его с помощью конструкции `using namespace`:

```
using namespace xml;  
Parser parser("myFile.xml");
```

- Лучше так не делать без крайней необходимости: это усложняет чтение кода и может приводить к конфликтам имён

Пространства имён

- Если пространство имён используется часто, можно подключить его с помощью конструкции `using namespace`:

```
using namespace xml;  
Parser parser("myFile.xml");
```

- Лучше так не делать без крайней необходимости: это усложняет чтение кода и может приводить к конфликтам имён
- В заголовочных файлах точно не стоит делать `using namespace`: оно неявно добавится во все файлы, использующие этот заголовочный файл

Пространства имён

- Если пространство имён используется часто, *можно* подключить его с помощью конструкции `using namespace`:

```
using namespace xml;  
Parser parser("myFile.xml");
```

- Лучше так не делать без крайней необходимости: это усложняет чтение кода и может приводить к конфликтам имён
- В заголовочных файлах точно не стоит делать `using namespace`: оно неявно добавится во все файлы, использующие этот заголовочный файл
- Совет: **не делайте** `using namespace std`;

Пространства имён

- Пространства имён могут быть вложенными:

```
namespace boost {  
    namespace multiprecision {  
        class gmp_int;  
    }  
}  
  
boost::multiprecision::gmp_int x(42);
```

Пространства имён

- Пространства имён могут быть вложенными:

```
namespace boost {  
    namespace multiprecision {  
        class gmp_int;  
    }  
}  
  
boost::multiprecision::gmp_int x(42);
```

- Для пространств имён можно создавать новые имена для удобства (это гораздо лучше, чем `using namespace`):

```
namespace bmp = boost::multiprecision;  
  
bmp::gmp_int x(42);
```

- Технически, всё, что не лежит в каком-то пространстве имён, лежит в неявном *глобальном пространстве имён*

Глобальное пространство имён

- Технически, всё, что не лежит в каком-то пространстве имён, лежит в неявном *глобальном пространстве имён*
- Обратиться к объектам из глобального пространства имён можно через оператор разрешения области видимости `::`

```
namespace complex {  
    class Complex { ... };  
  
    Complex exp(Complex c) {  
        float x = c.real;  
        float y = c.imag;  
  
        // ::exp(x) явно обращается к глобальному пространству имён  
        return ::exp(x) * ...;  
    }  
}
```

Анонимное пространство имён

- У пространства имён можно опустить имя: тогда оно будет *анонимным*

Анонимное пространство имён

- У пространства имён можно опустить имя: тогда оно будет *анонимным*
- Объекты из него автоматически попадают в родительское пространство имён, но используют *internal linkage* – не видны из других `cpp`-файлов при линковке

Анонимное пространство имён

- У пространства имён можно опустить имя: тогда оно будет *анонимным*
- Объекты из него автоматически попадают в родительское пространство имён, но используют *internal linkage* – не видны из других сpp-файлов при линковке
- То же самое, что и **static**, но удобнее, когда есть много объектов, и работает для классов:

```
static void helper();
static int global = 0;

namespace {
    void helper2();
    int global2 = 0;

    class Internal { ... };
}
```

Анонимное пространство имён

- У пространства имён можно опустить имя: тогда оно будет *анонимным*
- Объекты из него автоматически попадают в родительское пространство имён, но используют *internal linkage* – не видны из других `cpp`-файлов при линковке
- То же самое, что и `static`, но удобнее, когда есть много объектов, и работает для классов:

```
static void helper();
static int global = 0;

namespace {
    void helper2();
    int global2 = 0;

    class Internal { ... };
}
```

- Совет: все детали реализации внутри `cpp`-файла прячьте в анонимное пространство имён

- Пространство имён можно пометить как `inline`

Inline пространство имён

- Пространство имён можно пометить как `inline`
- Объекты из него автоматически попадают в родительское пространство имён, но всё равно используют имя пространства имён при name mangling'e

Inline пространство имён

- Пространство имён можно пометить как `inline`
- Объекты из него автоматически попадают в родительское пространство имён, но всё равно используют имя пространства имён при name mangling'e
- Иногда используется для версионирования библиотек (напр. libstdc++ – стандартная библиотека C++ компилятора GCC)

```
namespace mylib {  
    // Старый скомпилированный код использует v1  
    namespace v1 {  
        ...  
    }  
  
    // Новый код по умолчанию использует v2  
    inline namespace v2 {  
        ...  
    }  
}
```

- C-style синонимы типов `typedef int ObjectID;` часто тяжело воспринимаются

Синонимы типов

- C-style синонимы типов `typedef int ObjectID;` часто тяжело воспринимаются
- В C++ есть альтернативный синтаксис: `using ObjectID = int;`

СИНОНИМЫ ТИПОВ

- C-style синонимы типов `typedef int ObjectID;` часто тяжело воспринимаются
- В C++ есть альтернативный синтаксис: `using ObjectID = int;`
- Синонимы типов (как через `typedef`, так и через `using`) могут быть внутри классов или пространств имён:

```
namespace internal {  
    using Data = int;  
    struct Node {  
        Data data;  
        Node* next;  
    };  
}  
  
class List {  
    using Data = internal::Data;  
    using Node = internal::Node;  
    ...  
};
```

Стандартная библиотека C++

- Алгоритмы: `<algorithm>`
- Структуры данных: `<vector>`, `<list>`, `<forward_list>`,
`<deque>`, `<set>`, `<map>`, `<unordered_set>`, `<unordered_map>`
- Строки, поиск и форматирование: `<string>`, `<regex>`,
`<format>`, `<print>`
- Математика: `<cmath>`, `<complex>`, `<numeric>`
- Генерация случайных чисел: `<random>`
- Ввод/вывод: `<iostream>`, `<fstream>`, `<sstream>`
- Работа с памятью, умные указатели: `<new>`, `<memory>`,
`<memory_resource>`
- Работа с временем: `<chrono>`
- Многопоточное программирование: `<thread>`, `<mutex>`,
`<future>`, `<condition_variable>`
- И много другого

- Поддерживается любым компилятором C++

- Поддерживается любым компилятором C++
- Все идентификаторы находятся в пространстве имён `std`, напр. `std::string`, `std::regex`, `std::format`

- Поддерживается любым компилятором C++
- Все идентификаторы находятся в пространстве имён `std`, напр. `std::string`, `std::regex`, `std::format`
- Все имена используют `snake_case`

- Поддерживается любым компилятором C++
- Все идентификаторы находятся в пространстве имён `std`, напр. `std::string`, `std::regex`, `std::format`
- Все имена используют `snake_case`
- Заголовочные файлы не имеют расширения: `<algorithm>`, `<future>`

- Заголовочные файлы стандартной библиотеки C тоже доступны в C++: `<string.h>`, `<math.h>` и т.д.

Стандартная библиотека C++

- Заголовочные файлы стандартной библиотеки C тоже доступны в C++: `<string.h>`, `<math.h>` и т.д.
- В стандартной библиотеке C++ есть файлы, оборачивающие C-файлы в `namespace std` – рекомендуется использовать именно их

Стандартная библиотека C++

- Заголовочные файлы стандартной библиотеки C тоже доступны в C++: `<string.h>`, `<math.h>` и т.д.
- В стандартной библиотеке C++ есть файлы, оборачивающие C-файлы в `namespace std` – рекомендуется использовать именно их
- `<XXX.h>` превращается в `<cXXX>`: `<cstring>`, `<cmath>`, и т.д.

Стандартная библиотека C++

- Заголовочные файлы стандартной библиотеки C тоже доступны в C++: `<string.h>`, `<math.h>` и т.д.
- В стандартной библиотеке C++ есть файлы, оборачивающие C-файлы в `namespace std` – рекомендуется использовать именно их
- `<XXX.h>` превращается в `<cXXX>`: `<cstring>`, `<cmath>`, и т.д.
- Например, функция `sin(float)` из `<math.h>` превращается в функцию `std::sin(float)` из `<cmath>`

Стандартная библиотека C++

- Заголовочные файлы стандартной библиотеки C тоже доступны в C++: `<string.h>`, `<math.h>` и т.д.
- В стандартной библиотеке C++ есть файлы, оборачивающие C-файлы в `namespace std` – рекомендуется использовать именно их
- `<XXX.h>` превращается в `<cXXX>`: `<cstring>`, `<cmath>`, и т.д.
- Например, функция `sin(float)` из `<math.h>` превращается в функцию `std::sin(float)` из `<cmath>`
- Формально, `<cmath>` не обязан содержать функцию `sin` (без пространства имён `std`), но на практике часто содержит (но это не гарантируется!)

Стандартная библиотека C++

- Заголовочные файлы стандартной библиотеки C тоже доступны в C++: `<string.h>`, `<math.h>` и т.д.
- В стандартной библиотеке C++ есть файлы, оборачивающие C-файлы в `namespace std` – рекомендуется использовать именно их
- `<XXX.h>` превращается в `<cXXX>`: `<cstring>`, `<cmath>`, и т.д.
- Например, функция `sin(float)` из `<math.h>` превращается в функцию `std::sin(float)` из `<cmath>`
- Формально, `<cmath>` не обязан содержать функцию `sin` (без пространства имён `std`), но на практике часто содержит (но это не гарантируется!)
- То же самое касается типов: `<cstdint>` содержит тип `std::uint32_t` и не обязан содержать тип `uint32_t`, но на практике часто содержит, и многие не пишут `std::` перед такими типами

Пример: C-style сортировка

```
struct point {  
    int x, y;  
};  
  
void qsort(void* array, size_t count, size_t size,  
    int (*comp)(const void*, const void*));  
  
int cmp_point(const void* v1, const void* v2) {  
    const struct point *p1 = (const struct point*)v1;  
    const struct point *p2 = (const struct point*)v2;  
    ...  
}  
  
struct point points[10];  
qsort(points, 10, sizeof(points[0]), cmp_point);
```

Пример: C-style сортировка

```
struct point {  
    int x, y;  
};  
  
void qsort(void* array, size_t count, size_t size,  
    int (*comp)(const void*, const void*));  
  
int cmp_point(const void* v1, const void* v2) {  
    const struct point *p1 = (const struct point*)v1;  
    const struct point *p2 = (const struct point*)v2;  
    ...  
}  
  
struct point points[10];  
qsort(points, 10, sizeof(points[0]), cmp_point);
```

- Можно использовать и для примитивных типов

Пример: C-style сортировка

```
struct point {
    int x, y;
};

void qsort(void* array, size_t count, size_t size,
           int (*comp)(const void*, const void*));

int cmp_point(const void* v1, const void* v2) {
    const struct point *p1 = (const struct point*)v1;
    const struct point *p2 = (const struct point*)v2;
    ...
}

struct point points[10];
qsort(points, 10, sizeof(points[0]), cmp_point);
```

- Можно использовать и для примитивных типов
- Нет проверки типов со стороны компилятора (`void*`)

Пример: C-style сортировка

```
struct point {
    int x, y;
};

void qsort(void* array, size_t count, size_t size,
           int (*comp)(const void*, const void*));

int cmp_point(const void* v1, const void* v2) {
    const struct point *p1 = (const struct point*)v1;
    const struct point *p2 = (const struct point*)v2;
    ...
}

struct point points[10];
qsort(points, 10, sizeof(points[0]), cmp_point);
```

- Можно использовать и для примитивных типов
- Нет проверки типов со стороны компилятора (`void*`)
- Функцию-компаратор `cmp_point` невозможно заинлайнить

Пример: ООП-style сортировка

```
class Comparable {
public:
    virtual int compare(const Comparable* other) const = 0;
};

void sort(Comparable** array, size_t count);

class Point : public Comparable {
private:
    int x, y;
public:
    int compare(const Comparable* other) const override {
        const Point* otherPoint = (const Point*)other;
        ...
    }
};

Comparable* points[10];
points[0] = new Point(1, 2);
...
sort(points, 10);
```

Пример: ООП-style сортировка

- Проверка типов есть, но частичная – всё ещё приходится делать приведение типов `Comparable*` -> `Point*`

Пример: ООП-style сортировка

- Проверка типов есть, но частичная – всё ещё приходится делать приведение типов `Comparable*` -> `Point*`
- Метод-компаратор `compare` всё ещё невозможно заинлайнить

Пример: ООП-style сортировка

- Проверка типов есть, но частичная – всё ещё приходится делать приведение типов `Comparable* -> Point*`
- Метод-компаратор `compare` всё ещё невозможно заинлайнить
- Для примитивных типов приходится писать обёртки

```
class Integer : public Comparable {  
private:  
    int value;  
public:  
    int compare(const Comparable* other) const override {  
        ...  
    }  
};
```

Пример: ООП-style сортировка

- Приходится хранить объекты по указателю

Пример: ООП-style сортировка

- Приходится хранить объекты по указателю
- Сработает ли такой код?

```
void sort(Comparable* array, size_t count);
```

```
Point points[10];  
sort(points, 10);
```

Пример: ООП-style сортировка

- Приходится хранить объекты по указателю
- Сработает ли такой код?

```
void sort(Comparable* array, size_t count);  
  
Point points[10];  
sort(points, 10);
```

- Нет: при итерации по массиву функции `sort` нужно знать настоящий размер объектов
(`sizeof(Point) > sizeof(Comparable)`)

Пример: C-style дерево поиска

```
struct node {
    void* data;
    struct node *left, *right;
};

struct tree {
    struct node* root;
};

typedef int (*cmp_fun)(const void*, const void*);

void insert(struct tree* t, void* data, cmp_fun cmp) {
    struct node* n = malloc(sizeof(struct node));
    ...
}

struct list l; l.head = NULL;
int* data = malloc(sizeof(int));
*data = 42;
insert(&l, data, cmp_int);
```

Пример: C-style дерево поиска

```
struct node {
    void* data;
    struct node *left, *right;
};

struct tree {
    struct node* root;
};

typedef int (*cmp_fun)(const void*, const void*);

void insert(struct tree* t, void* data, cmp_fun cmp) {
    struct node* n = malloc(sizeof(struct node));
    ...
}

struct list l; l.head = NULL;
int* data = malloc(sizeof(int));
*data = 42;
insert(&l, data, cmp_int);
```

- Нет проверки типов, функцию-компаратор не заинлайнить

Пример: C-style дерево поиска

```
struct node {  
    void* data;  
    struct node *left, *right;  
};  
  
struct tree {  
    struct node* root;  
};  
  
typedef int (*cmp_fun)(const void*, const void*);  
  
void insert(struct tree* t, void* data, cmp_fun cmp) {  
    struct node* n = malloc(sizeof(struct node));  
    ...  
}  
  
struct list l; l.head = NULL;  
int* data = malloc(sizeof(int));  
*data = 42;  
insert(&l, data, cmp_int);
```

- Нет проверки типов, функцию-компаратор не заинлайнить
- Две аллокации памяти!

Пример: ООП-style дерево поиска

```
class Tree {  
    struct Node {  
        Comparable* data;  
        Node *left, *right;  
    };  
  
    Node* root;  
  
public:  
  
    void insert(Comparable* data) {  
        Node* node = new Node;  
        ...  
    }  
};  
  
Tree tree;  
tree.insert(new Point(3, 4));
```

Пример: ООП-style дерево поиска

```
class Tree {  
    struct Node {  
        Comparable* data;  
        Node *left, *right;  
    };  
  
    Node* root;  
  
public:  
  
    void insert(Comparable* data) {  
        Node* node = new Node;  
        ...  
    }  
};  
  
Tree tree;  
tree.insert(new Point(3, 4));
```

- Все те же проблемы, что и с сортировкой

- Все проблемы в примерах исчезли бы, если бы мы умели *параметризовать* функции/классы некоторыми *типами*

- Все проблемы в примерах исчезли бы, если бы мы умели *параметризовать* функции/классы некоторыми *типами*
- Сейчас мы умеем так:

```
// Приходится писать новый класс для
// каждого хранимого типа данных
class Array {
    int* data;
    size_t size;
};

// Приходится писать новый класс для
// каждого хранимого типа данных
class scoped_ptr {
    int* data;
};
```

- А хочется как-то так:

```
class Array {  
    // T - как-то заданный, но произвольный тип  
    T* data;  
    size_t size;  
};  
  
class scoped_ptr {  
    // T - как-то заданный, но произвольный тип  
    T* data;  
};
```

Шаблоны на С препроцессоре

- Решение 1: использовать препроцессор

```
#define Array(T) class Array_##T { \
private: \
    T* data; \
    size_t size; \
public: \
    T& operator[](size_t index) { \
        return array[index]; \
    } \
    ... \
};

Array(int) // генерируем код класса Array_int
Array(float) // генерируем код класса Array_float

Array_int array(10);
array[5] = 42;
```

- Есть все проблемы от препроцессора: простая текстовая подстановка, неудобство отладки, и т.п.

- Есть все проблемы от препроцессора: простая текстовая подстановка, неудобство отладки, и т.п.
- Классы `Array_int` и `Array_float` никак не связаны между собой

- Решение 2: использовать C++ шаблоны

- Решение 2: использовать C++ шаблоны

```
template <typename T>
class Array {
private:
    T* data;
    size_t size;
public:
    T& operator[](size_t index) {
        return array[index];
    }
    ...
};

Array<int> array(10);
array[5] = 42;
```

- Подстановку делает не препроцессор, а компилятор – он знает про типы, про области видимости, и про другие конструкции языка

- Подстановку делает не препроцессор, а компилятор – он знает про типы, про области видимости, и про другие конструкции языка
- Работает посредством *мономорфизации*: компилятор внутри фактически генерирует отдельные классы `Array<int>`, `Array<float>` и т.д.

- Подстановку делает не препроцессор, а компилятор – он знает про типы, про области видимости, и про другие конструкции языка
- Работает посредством *мономорфизации*: компилятор внутри фактически генерирует отдельные классы `Array<int>`, `Array<float>` и т.д.
- Код шаблонного класса проще читать, и можно разбить на несколько файлов при желании (но всё равно должен быть в заголовочном файле)

Шаблоны в C++: примеры

```
template <typename T>
class scoped_ptr {
    T* data;
    ...
};
```

```
template <typename T>
class list {
    struct node {
        T data;
        node* next;
    };

    node* head;
    ...
};
```

```
template <typename T>
class tree {
    struct node {
        T data;
        node *left, *right;
    };

    node* root;
    ...
};
```

```
template <typename T>
void sort(T* array,
         size_t count) {
    ...
}
```